

CV Systems Engineering

Architecture, Data Flow & PDF Generation Logic

SYSTEM ARCHITECTURE

Data Independence

The CV content is decoupled from the visual representation. By storing data in **cv.json**, we treat information as a variable, not a static text.

- Single source of truth.
- Scalable for multiple languages.
- Conditional rendering (Active/Inactive).

Multi-Paradigm Flow

This approach allows us to demonstrate different programming styles as required by the project instructions.

REQUIRED CONTENT STRUCTURE

Category Type	IT Context (Programmer Tab)	Actor Context (Actress Tab)
Core Identification	Software Engineer Profile	Acting Portfolio / Headshot
Experience	Technical Projects / Internships	Theater & Film Credits
Skillset	C++, Python, Data Analysis	Stage Combat, Dialects, Dance
Validation	Education & Tech Certs	Workshops & Acting Schools

HOW PDF CONVERSION WORKS

The conversion process follows a **Mapping Pattern**:

- 1. Trigger:** User clicks the HTML Button.
- 2. Parsing:** Code reads the `cv.json` file.
- 3. Filtering:** Code checks if `(item.active == true)`.
- 4. Rendering:** Data is injected into a LaTeX or HTML template.
- 5. Output:** A Blob (Binary Large Object) is created and downloaded.

```
// Conceptual Logic function generatePDF() {  
  const data = loadJSON('cv.json'); const  
  activeItems = data.filter(i => i.active); const  
  doc = new PDFGenerator();  
  doc.addText(activeItems);  
  doc.save('Nissu_Pagan_CV.pdf'); }
```


CODE-DRIVEN GENERATION

The Project Requirement: "You must generate a PDF from your code."

This means we cannot just use the browser's "Print to PDF" menu. We must use a library (like `html2pdf`, `jsPDF`, or a Python script) that constructs the file binary from the raw data.

INTERFACE MOCKUP



APPLYING PROGRAMMING PARADIGMS

Imperative

Using **PyScript** to loop through JSON entries and manually append them to a document object.

Object Oriented

Defining a CVSection class where each category inherits properties like visibility and render().

THE LATEX CONNECTION

How we use your basic LaTeX knowledge:

- **Template:** Design the layout in Overleaf to get the `.tex` code.
- **Placeholders:** Replace text with variables like `{{NAME}}`.
- **Compiler:** Our program replaces placeholders with `cv.json` data and sends it to a generator.



JSON Data + LaTeX Template = Professional PDF

PROJECT REQUIREMENTS AUDIT

- ✓ 10 Categories programmed.
- ✓ Input/Output via JSON storage.
- ✓ Print resume in HTML method.
- ✓ Generate PDF via Download Button.
- ✓ Toggle entries (active/inactive) logic.

IMPLEMENTATION ROADMAP

1

Finalize 10 categories in
`cv.json`

2

Add "Download" button to
Programmer Tab

3

Inject `html2pdf.js` library

4

Write the paradigm-specific
logic



Questions?

Ready to code the Download function?